

I hadn't used [D3.js](#) before, so I gave gemini the steps that we needed to accomplish and asked it how do I go about it. If I could get a quick and dirty example about how I could create my website. The code Gemini gave did not work however and I needed to tweak it to parse the csv appropriately. After that I needed to remove all the unnecessary fix the linegraphs that gemini had produced as they were still only parsing for a single temperature value. Then I fixed the color coding of the graphs to better represent the temperatures as initially, the color coding for the "minimum temperature" mode denoted the colder temperature as lighter colors. After that I needed to fix a bug with the hover tooltip appearing behind the graphs and then I modularized everything by putting them into separate files.

I haven't used [Dr.js](#) before. I have to try and do these steps:

In the matrix, the x direction indicates the year, and the y direction indicates the month. Each cell indicates the corresponding month of a specific year.

1. You need to visualize the maximum and minimum temperatures by month using the background color of each cell. Please use a mouse click to switch between the maximum and minimum temperatures.
2. When hovering the mouse on each cell, a tip should appear to show the date and the temperature value.
3. Add a mini line chart to each cell to show daily temperature changes. The x-axis represents the days in a month, and the y-axis represents the temperature.
4. A legend is needed to show the mapping between colors and values.

How do I do so, give me a quick and dirty example that could work with this

I have written a complete, all-in-one [index.html](#) template. It includes the CSS, the D3.js setup, the SVG matrix, the mini line charts, the tooltip, the click-to-toggle feature, and the legend.

What you need to do:

1. Save the code below as an [index.html](#) file.
2. Ensure your [temperature_daily.csv](#) is in the **same folder**.

3. *Crucial:* Check your CSV column names. The code below assumes your CSV has standard columns: `date` (YYYY-MM-DD) and `temperature`. If your columns are named differently (like `Temp` or `Date`), simply update the `d3.csv` parsing block at the top of the script.
4. Run it on a local web server (e.g., using VS Code's Live Server extension or running `python -m http.server` in your terminal).

The Code

HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>HK Temperature Matrix View</title>
```

```
  <script src="https://d3js.org/d3.v7.min.js"></script>
```

```
<style>
```

```
  body { font-family: sans-serif; text-align: center; }
```

```
  #chart-container { position: relative; display: inline-block; margin-top: 20px; }
```

```
  /* Tooltip Styling */
```

```
  .tooltip {
```

```
    position: absolute;
```

```
    background: rgba(255, 255, 255, 0.95);
```

```
    border: 1px solid #ccc;
```

```
    padding: 10px;
```

```
    pointer-events: none;
```

```
    opacity: 0;
```

```
    box-shadow: 2px 2px 5px rgba(0,0,0,0.1);  
    font-size: 14px;  
    transition: opacity 0.2s;  
}
```

```
/* Mini Line Chart Styling */
```

```
.sparkline {  
    fill: none;  
    stroke: #333;  
    stroke-width: 1.5px;  
    opacity: 0.8;  
}
```

```
/* Toggle Button */
```

```
button {  
    padding: 10px 20px;  
    font-size: 16px;  
    cursor: pointer;  
    background-color: #4CAF50;  
    color: white;  
    border: none;  
    border-radius: 5px;  
    margin-bottom: 10px;  
}
```

```
button:hover { background-color: #45a049; }
```

```
text { font-size: 12px; fill: #333; }
```

```
.domain, .tick line { stroke: #ccc; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Hong Kong Monthly Temperature (Last 10 Years)</h2>
```

```
<p>Click the button below or a cell to toggle between Maximum and Minimum  
Temperatures.</p>
```

```
<button id="toggle-btn">Showing: MAX Temperature</button>
```

```
<br>
```

```
<div id="chart-container">
```

```
  <div id="tooltip" class="tooltip"></div>
```

```
</div>
```

```
<script>
```

```
  // --- 1. SETUP DIMENSIONS ---
```

```
  const margin = { top: 50, right: 100, bottom: 50, left: 80 };
```

```
  const width = 1000 - margin.left - margin.right;
```

```
  const height = 600 - margin.top - margin.bottom;
```

```
  const svg = d3.select("#chart-container")
```

```
    .append("svg")
```

```
.attr("width", width + margin.left + margin.right)

.attr("height", height + margin.top + margin.bottom)

.append("g")

.attr("transform", `translate(${margin.left},${margin.top})`);
```

```
// State variable for toggle
```

```
let currentMode = "max"; // or "min"
```

```
// --- 2. LOAD & PROCESS DATA ---
```

```
d3.csv("temperature_daily.csv").then(rawData => {

  const parseDate = d3.timeParse("%Y-%m-%d");
```

```
// Format data (Update column names here if your CSV differs!)
```

```
let data = rawData.map(d => {

  const dateObj = parseDate(d.date);

  return {

    date: dateObj,

    year: dateObj.getFullYear(),

    month: dateObj.getMonth() + 1, // 1-12

    day: dateObj.getDate(),

    temp: +d.temperature // Ensure this matches your CSV column

  };

}).filter(d => !isNaN(d.temp) && d.date !== null);
```

```
// Filter last 10 years
```

```
const maxYear = d3.max(data, d => d.year);
```

```
data = data.filter(d => d.year > maxYear - 10);
```

```
// Group data by Year -> Month
```

```
// Calculate max/min and preserve daily array for the line chart
```

```
const monthlyData = [];
```

```
const grouped = d3.group(data, d => d.year, d => d.month);
```

```
grouped.forEach((months, year) => {
```

```
  months.forEach((days, month) => {
```

```
    days.sort((a, b) => d3.ascending(a.day, b.day));
```

```
    monthlyData.push({
```

```
      year: year,
```

```
      month: month,
```

```
      maxTemp: d3.max(days, d => d.temp),
```

```
      minTemp: d3.min(days, d => d.temp),
```

```
      daily: days // Array of daily objects for sparkline
```

```
    });
```

```
  });
```

```
});
```

```
// --- 3. SCALES ---
```

```
const years = Array.from(new Set(monthlyData.map(d => d.year))).sort();
```

```
const months = d3.range(1, 13);
```

```
const xScale = d3.scaleBand()
```

```
  .domain(years)
```

```
  .range([0, width])
```

```
  .padding(0.05);
```

```
const yScale = d3.scaleBand()
```

```
  .domain(months)
```

```
  .range([0, height])
```

```
  .padding(0.05);
```

```
const cellWidth = xScale.bandwidth();
```

```
const cellHeight = yScale.bandwidth();
```

```
// Color scales (Red/Orange for Max, Blue/Teal for Min)
```

```
const maxTempExtent = d3.extent(monthlyData, d => d.maxTemp);
```

```
const minTempExtent = d3.extent(monthlyData, d => d.minTemp);
```

```
const colorScaleMax =  
d3.scaleSequential(d3.interpolateYlOrRd).domain([maxTempExtent[0], maxTempExtent[1]]);
```

```
const colorScaleMin =  
d3.scaleSequential(d3.interpolateBlues).domain([minTempExtent[0], minTempExtent[1]]);
```

```
// Mini chart scales
```

```

const miniX = d3.scaleLinear().domain([1, 31]).range([2, cellWidth - 2]);

const miniY = d3.scaleLinear().domain([d3.min(data, d=>d.temp), d3.max(data,
d=>d.temp)]).range([cellHeight - 2, 2]);

const lineGenerator = d3.line()

  .x(d => miniX(d.day))

  .y(d => miniY(d.temp));

// --- 4. DRAW AXES ---

svg.append("g").call(d3.axisTop(xScale).tickFormat(d3.format("d")));

const monthNames = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct",
"Nov", "Dec"];

svg.append("g").call(d3.axisLeft(yScale).tickFormat(d => monthNames[d - 1]));

// --- 5. DRAW MATRIX CELLS ---

const tooltip = d3.select("#tooltip");

const cells = svg.selectAll(".cell-group")

  .data(monthlyData)

  .enter()

  .append("g")

  .attr("class", "cell-group")

  .attr("transform", d => `translate(${xScale(d.year)}, ${yScale(d.month)})`)

  .on("mouseover", (event, d) => {

```



```

        tooltip.style("opacity", 1);

        const val = currentMode === "max" ? d.maxTemp : d.minTemp;

        tooltip.html(`
            <strong>${monthNames[d.month-1]} ${d.year}</strong><br/>
            ${currentMode.toUpperCase()} Temp: ${val.toFixed(1)}°C
        `);
    })

    .on("mousemove", event => {
        tooltip.style("left", (event.pageX + 15) + "px")
            .style("top", (event.pageY - 20) + "px");
    })

    .on("mouseout", () => tooltip.style("opacity", 0))

    .on("click", toggleMode); // Fulfills click-to-switch requirement

// Add Rectangles (Background color)

const rects = cells.append("rect")

    .attr("width", cellWidth)

    .attr("height", cellHeight)

    .attr("rx", 3)

    .attr("fill", d => colorScaleMax(d.maxTemp));

// Add Mini Line Charts

cells.append("path")

    .attr("class", "sparkline")

```

```

    .datum(d => d.daily)

    .attr("d", lineGenerator);

// --- 6. LEGEND ---

// A simple placeholder for the legend. For a true gradient legend in 30 mins,
// we will draw a color bar based on the current scale.

const legendGrp = svg.append("g").attr("transform", `translate(${width + 20}, 20)`);

function drawLegend() {

    legendGrp.selectAll("*").remove(); // Clear old

    const scale = currentMode === "max" ? colorScaleMax : colorScaleMin;

    const domain = scale.domain();

    legendGrp.append("text").attr("y", -10).text(currentMode === "max" ? "Max °C" : "Min
    °C").style("font-weight", "bold");

    // Draw a simple 10-step discrete color bar as legend

    const steps = 10;

    const stepHeight = 20;

    const legendData = d3.range(domain[1], domain[0], -(domain[1]-domain[0])/steps);

    legendGrp.selectAll("rect")

        .data(legendData)

        .enter().append("rect")

```

```

        .attr("y", (d, i) => i * stepHeight)

        .attr("width", 20)

        .attr("height", stepHeight)

        .attr("fill", d => scale(d));

    legendGrp.selectAll("text.label")

        .data(legendData)

        .enter().append("text")

        .attr("class", "label")

        .attr("x", 25)

        .attr("y", (d, i) => i * stepHeight + 15)

        .text(d => Math.round(d));
    }

    drawLegend();

// --- 7. INTERACTION / TOGGLE ---

function toggleMode() {

    currentMode = currentMode === "max" ? "min" : "max";

    // Update Button Text

    d3.select("#toggle-btn").text(`Showing: ${currentMode.toUpperCase()} Temperature`);

    // Animate background color change

```

```

        rects.transition().duration(500)

            .attr("fill", d => currentMode === "max" ? colorScaleMax(d.maxTemp) :
colorScaleMin(d.minTemp));

        // Redraw legend

        drawLegend();

    }

    d3.select("#toggle-btn").on("click", toggleMode);

    }).catch(err => {

        console.error("Error loading CSV data. Check file path and column names.", err);

        d3.select("body").append("h3").style("color", "red").text("Failed to load
temperature_daily.csv. Please check the browser console.");

    });

</script>

</body>

</html>

```

Show me how you're reading the temp? where is that being parsed?

Gemini said

The temperature is being read and parsed right inside the `d3.csv(...).then(...)` block, specifically within the `.map()` function that loops through your raw data.

Here is the exact snippet where it happens:

JavaScript

```
let data = rawData.map(d => {  
  const dateObj = parseDate(d.date);  
  return {  
    date: dateObj,  
    year: dateObj.getFullYear(),  
    month: dateObj.getMonth() + 1,  
    day: dateObj.getDate(),  
    temp: +d.temperature // <--- RIGHT HERE  
  };  
})
```

Breaking down what that does:

1. **d3.csv(...)** loads everything as a string. When D3 reads your CSV, it treats every cell as text (e.g., "28.5" instead of the number 28.5).
2. **d.temperature** targets the column. It looks at the current row (**d**) and pulls the value from the column exactly named **temperature**. **(This is case-sensitive!)**
3. **The + sign does the parsing.** The **+** right before **d.temperature** is a JavaScript shortcut (the unary plus operator). It forces that text string to convert into an actual JavaScript number. Without the **+**, D3 wouldn't be able to calculate the maximums or minimums correctly because it would be trying to do math on words instead of numbers.

You said

So slight problem, the csv file doesn't deal in just a temperature column, it has a max_temperature of that day and a min_temperature. i'm fixing the loading part so it will look like this, right?

```
d3.csv("temperature_daily.csv").then(rawData => {  
  
  const parseDate = d3.timeParse("%Y-%m-%d");  
  
  // Format data (Update column names here if your CSV differs!)  
  
  let data = rawData.map(d => {  
  
    const dateObj = parseDate(d.date);  
  
    return {  
  
      date: dateObj,  
  
      year: dateObj.getFullYear(),  

```

```

month: dateObj.getMonth() + 1, // 1-12

day: dateObj.getDate(),

//get max_temperature and min_temperature from the CSV, adjust column names if needed
max_temp: +d.max_temperature, // Change to min_temperature for min mode
min_temp: +d.min_temperature

};

})//.filter(d => !isNaN(d.temp) && d.date != null);

// Filter last 10 years

const maxYear = d3.max(data, d => d.year);

data = data.filter(d => d.year > maxYear - 10);

Now how would the grouping look like

How do we fix this part?

grouped.forEach((months, year) => {

  months.forEach((days, month) => {

    days.sort((a, b) => d3.ascending(a.day, b.day));

    monthlyData.push({

      year: year,

      month: month,

      maxTemp: d3.max(days, d => d.temp),

      minTemp: d3.min(days, d => d.temp),

      daily: days // Array of daily objects for sparkline

    });

  });

});

});

```

Just answer this part

Gemini said

You are spot on with your parsing fix! Since you now have daily maximums and daily minimums, calculating the monthly extremes means you need to find the **max of the daily maxes** and the **min of the daily mins**.

Here is exactly how you fix that grouping block:

JavaScript

```
grouped.forEach((months, year) => {
  months.forEach((days, month) => {
    days.sort((a, b) => d3.ascending(a.day, b.day));
    monthlyData.push({
      year: year,
      month: month,
      // Find the highest max_temp of the month
      maxTemp: d3.max(days, d => d.max_temp),
      // Find the lowest min_temp of the month
      minTemp: d3.min(days, d => d.min_temp),
      daily: days // Array of daily objects for sparkline
    });
  });
});
```

One quick heads-up for your next step: Because you changed `d.temp` to `d.max_temp` and `d.min_temp`, you will also need to update the `lineGenerator` (the mini line chart) further down in the code so it knows which of the two variables to draw for the daily changes!

Would you like me to show you how to update the mini line chart to handle these new variables?

You said

what's the sorting going on over there? what are we sorting?

Gemini said

We are sorting the **days of the month in chronological order** (from the 1st to the 31st).

Here is the exact line you are asking about: `days.sort((a, b) => d3.ascending(a.day, b.day));`

What exactly is happening here?

Because D3 grouped your data first by year, then by month, the `days` variable holds an array of all the daily temperature records for one specific month (e.g., all of January 2020).

However, depending on how your CSV was originally generated, those rows might not be in perfect chronological order. `d3.ascending(a.day, b.day)` compares the `day` value (1 through 31) of two records and forces the array to sort them from lowest to highest.

Why does this matter?

It is strictly for the mini line chart (sparkline). When D3 draws a line chart, it connects point A to point B to point C in the exact order they appear in the array. If your days are out of order (for example: the 1st, then the 15th, then the 2nd), the line will zigzag violently backwards and forwards across the matrix cell instead of drawing a smooth, left-to-right timeline.

Since you changed your variables to `max_temp` and `min_temp`, those mini line charts are currently broken anyway because they don't know which one to draw.

Would you like me to give you the exact lines to fix the mini line chart so it toggles perfectly with your new max/min variables?

You said

Alright what do we fix here:

```
const years = Array.from(new Set(monthlyData.map(d => d.year))).sort();
```

```
const months = d3.range(1, 13);
```

```
const xScale = d3.scaleBand()
```

```
.domain(years)
```

```
.range([0, width])
```

```
.padding(0.05);
```

```
const yScale = d3.scaleBand()
```

```
.domain(months)
```

```
.range([0, height])
```

```
.padding(0.05);
```

```
const cellWidth = xScale.bandwidth();
```



```

const cellHeight = yScale.bandwidth();

// Color scales (Red/Orange for Max, Blue/Teal for Min)

const maxTempExtent = d3.extent(monthlyData, d => d.maxTemp);

const minTempExtent = d3.extent(monthlyData, d => d.minTemp);

const colorScaleMax = d3.scaleSequential(d3.interpolateYlOrRd).domain([maxTempExtent[0],
maxTempExtent[1]]);

const colorScaleMin = d3.scaleSequential(d3.interpolateBlues).domain([minTempExtent[0],
minTempExtent[1]]);

// Mini chart scales

const miniX = d3.scaleLinear().domain([1, 31]).range([2, cellWidth - 2]);

const miniY = d3.scaleLinear().domain([d3.min(data, d=>d.temp), d3.max(data,
d=>d.temp)]).range([cellHeight - 2, 2]);

const lineGenerator = d3.line()

.x(d => miniX(d.day))

.y(d => miniY(d.temp));

hint?

```

Gemini said

You asked for a hint, so I won't give you the copy-paste answer just yet!

Look closely at the bottom of that snippet, specifically in the **Mini chart scales** and the **lineGenerator**.

Here is your hint:

You have a ghost variable haunting your code. In three different places, you are still calling `d.temp`.

Remember, we just deleted `d.temp` in the loading phase! Your daily data now uses `d.max_temp` and `d.min_temp`. Because D3 can't find `d.temp` anymore, those mini line charts are going to break or disappear.

To fix it, you need to answer two questions:

1. **For miniY:** To set the overall y-axis bounds for the mini charts, you need the absolute lowest temperature and the absolute highest temperature in your entire dataset. Which variables should replace `d.temp` inside `d3.min()` and `d3.max()`?
2. **For lineGenerator:** Now that you have both daily maximums and daily minimums, what should the mini line chart actually draw?

Would you like the line chart to dynamically switch between the max and min lines when the user clicks the toggle button, or would you rather I show you how to draw *both* lines on the same mini chart at the same time?

You said

So the chart needs to draw two lines, not one. It needs to draw a line for min and max per day. Also, we might not have data for every day so we should allow for interpolation, is that handled?

Would it look something like this?

```
// --- 3. SCALES ---
```

```
const years = Array.from(new Set(monthlyData.map(d => d.year))).sort();
```

```
const months = d3.range(1, 13);
```

```
const xScale = d3.scaleBand()
```

```
.domain(years)
```

```
.range([0, width])
```

```
.padding(0.05);
```

```
const yScale = d3.scaleBand()
```

```
.domain(months)
```

```
.range([0, height])
```

```
.padding(0.05);
```

```
const cellWidth = xScale.bandwidth();
```

```
const cellHeight = yScale.bandwidth();
```

```
// Color scales (Red/Orange for Max, Blue/Teal for Min)
```

```
const maxTempExtent = d3.extent(monthlyData, d => d.maxTemp);
```

```

const minTempExtent = d3.extent(monthlyData, d => d.minTemp);

const colorScaleMax = d3.scaleSequential(d3.interpolateYlOrRd).domain([maxTempExtent[0],
maxTempExtent[1]]);

const colorScaleMin = d3.scaleSequential(d3.interpolateBlues).domain([minTempExtent[0],
minTempExtent[1]]);

// Mini chart scales

const miniX = d3.scaleLinear().domain([1, 31]).range([2, cellWidth - 2]);

const miniY = d3.scaleLinear().domain([d3.min(data, d=>d.min_temp), d3.max(data,
d=>d.max_temp)]).range([cellHeight - 2, 2]);

const lineGenerator1 = d3.line()

.x(d => miniX(d.day))

.y(d => miniY(d.max_temp)); // Use max_temp for line chart

const lineGenerator2 = d3.line()

.x(d => miniX(d.day))

.y(d => miniY(d.min_temp)); // Use min_temp for line chart

```

Gemini said

You nailed the logic! Your `miniY` scale correctly finds the absolute lowest minimum and absolute highest maximum to set the bounds, and your two line generators are perfectly mapped.

Regarding missing data and interpolation: If a day is completely missing from your CSV (e.g., it skips from day 2 directly to day 4), D3 will naturally draw a straight line from day 2 to day 4. That's standard interpolation.

However, if the row *does* exist but the temperature value is blank or `NaN`, D3 will try to draw a coordinate at `NaN` and the entire line will break and disappear. To prevent this, you should use D3's `.defined()` method. This tells the line generator, "Only draw a point if the data is actually a valid number; otherwise, skip it and connect to the next valid point."

Here is how your code should look with `.defined()` added:

JavaScript

```

// Mini chart scales
const miniX = d3.scaleLinear().domain([1, 31]).range([2, cellWidth - 2]);
const miniY = d3.scaleLinear()
  .domain([d3.min(data, d => d.min_temp), d3.max(data, d => d.max_temp)])
  .range([cellHeight - 2, 2]);

// Max Line Generator
const lineGenerator1 = d3.line()
  .defined(d => d.max_temp !== null && !isNaN(d.max_temp)) // Protects against missing
data
  .x(d => miniX(d.day))
  .y(d => miniY(d.max_temp));

// Min Line Generator
const lineGenerator2 = d3.line()
  .defined(d => d.min_temp !== null && !isNaN(d.min_temp)) // Protects against missing
data
  .x(d => miniX(d.day))
  .y(d => miniY(d.min_temp));

```

Next Step: Drawing Both Lines

Since you now have two line generators, you will need to append two separate `<path>` elements inside your matrix cells instead of one.

When you get to **Step 5 (DRAW MATRIX CELLS)** in the original template, you'll want to replace the single `cells.append("path")` block with this:

JavaScript

```

// Add Max Temperature Mini Line
cells.append("path")
  .attr("class", "sparkline")
  .datum(d => d.daily)
  .attr("d", lineGenerator1)
  .style("stroke", "red") // Optional: color-code the lines
  .style("stroke-width", "1.5px")
  .style("fill", "none");

// Add Min Temperature Mini Line
cells.append("path")
  .attr("class", "sparkline")
  .datum(d => d.daily)
  .attr("d", lineGenerator2)

```

```
.style("stroke", "blue") // Optional: color-code the lines
.style("stroke-width", "1.5px")
.style("fill", "none");
```

Would you like me to review the tooltip section next to ensure it toggles smoothly between displaying the Max and Min values when you click the cells?

You said

I think everything works fine but the color scale is a little confusing, The high temperature makes sense but I want the colder temperatures to be darker shades of blue

Gemini said

That is a great catch. From a data visualization standpoint, it is much more intuitive for "colder" to equal a heavier, darker blue!

By default, `d3.interpolateBlues` maps the lower number in your domain to the lightest color (almost white) and the higher number to the darkest blue.

To flip that so the lowest temperatures get the darkest blue, all you have to do is **reverse the order of the domain** for `colorScaleMin`.

Here is the exact line to change:

JavaScript

// BEFORE:

```
const colorScaleMin = d3.scaleSequential(d3.interpolateBlues).domain([minTempExtent[0],
minTempExtent[1]]);
```

// AFTER (Flipped domain):

```
const colorScaleMin = d3.scaleSequential(d3.interpolateBlues).domain([minTempExtent[1],
minTempExtent[0]]);
```

Why this works:

By putting `minTempExtent[1]` (the highest minimum temp) first, D3 maps it to the start of the color scheme (light blue). Then, `minTempExtent[0]` (the absolute coldest temp) is mapped to the end of the color scheme (dark blue).

Since flipping the domain might invert the order of the numbers drawn in your legend for the minimum temperatures, would you like me to show you a quick tweak for the `drawLegend()` function to keep the numbers reading from highest to lowest?

You said

Cool, but now for the legend display, i want the cold legend to be upside down because currently, 3 degrees is at the top and 26 degrees at the bottom

Gemini said

That is exactly what happens when you flip the domain for the colors! Because the legend builds itself based on the order of the domain, it accidentally flipped your numbers upside down, putting the coldest temperature at the top.

To fix this, we just need to force the legend to *always* draw from the highest number down to the lowest number, regardless of how the color domain is ordered.

Here is the quick fix for your `drawLegend()` function. Replace the old `legendData` logic with this:

JavaScript

```
function drawLegend() {
  legendGrp.selectAll("*").remove(); // Clear old

  const scale = currentMode === "max" ? colorScaleMax : colorScaleMin;
  const domain = scale.domain();

  legendGrp.append("text")
    .attr("y", -10)
    .text(currentMode === "max" ? "Max °C" : "Min °C")
    .style("font-weight", "bold");

  // 1. Find the absolute highest and lowest values of the current mode
  const highest = Math.max(domain[0], domain[1]);
  const lowest = Math.min(domain[0], domain[1]);

  // 2. Always generate the legend data from highest down to lowest
  const steps = 10;
  const stepHeight = 20;
  const stepSize = (highest - lowest) / steps;

  // We subtract a tiny bit from 'lowest' to ensure the final step is included
  const legendData = d3.range(highest, lowest - (stepSize / 2), -stepSize);
```

```

    legendGrp.selectAll("rect")
      .data(legendData)
      .enter().append("rect")
      .attr("y", (d, i) => i * stepHeight)
      .attr("width", 20)
      .attr("height", stepHeight)
      .attr("fill", d => scale(d)); // This still correctly matches your flipped colors!

    legendGrp.selectAll("text.label")
      .data(legendData)
      .enter().append("text")
      .attr("class", "label")
      .attr("x", 25)
      .attr("y", (d, i) => i * stepHeight + 15)
      .text(d => Math.round(d));
  }

```

Why this works:

By extracting `highest` and `lowest` using `Math.max()` and `Math.min()`, we ignore the reversed order of the `minTempExtent`. `d3.range(highest, lowest, -stepSize)` guarantees your array always starts at the big number (like 26°C) and steps down to the small number (like 3°C), putting the warmest temperatures safely at the top of your legend!

You are super close to having this entirely wrapped up. Would you like me to do a final review of all the integrated parts, or are you ready to run it and beat that 30-minute deadline?

You said

The hovering textbox feels weird, it's all the way to the right, why?

Gemini said

Ah, the classic D3 tooltip offset glitch! I know exactly why that is happening.

It's a conflict between your CSS and your JavaScript coordinates.

In the HTML, the tooltip `<div>` is sitting *inside* your `#chart-container`. Because the container is centered on the page and has `position: relative`, the tooltip calculates its position based on the container. However, JavaScript's `event.pageX` measures the mouse position from the far left edge of your *entire screen*.

Basically, it's taking the center of your page and then adding the entire screen's width to it, launching your tooltip way off to the right!

The Quickest Fix

You don't even need to touch the JavaScript. Just move the tooltip `<div>` outside of the chart container so it sits directly inside the `<body>`. That way, the screen coordinates and the tooltip coordinates will match perfectly.

Find this part in your HTML:

HTML

```
<div id="chart-container">
  <div id="tooltip" class="tooltip"></div> </div>
```

And change it to this:

HTML

```
<div id="tooltip" class="tooltip"></div> <div id="chart-container">
  </div>
```

Save and refresh. Your tooltip should now stick perfectly to your mouse cursor!

How is the rest of the matrix looking? Any other weird formatting bugs popping up before you submit this?

You said

Uhh ok, now the textbox seems to be behind the matrix of the minigraphs

Gemini said

Ah, the classic CSS stacking order strikes again!

When you moved the tooltip outside of the container, it lost its priority in the browser's 3D stacking order. Now, the browser is drawing the massive SVG matrix *over* the top of your tooltip.

The Fix: Add a Z-Index

You just need to tell the CSS to force the tooltip to the very front of the screen.

Scroll up to the `<style>` block in your HTML head, find the `.tooltip` class, and add `z-index: 1000;` to it. It should look exactly like this:

CSS

```
/* Tooltip Styling */
.tooltip {
  position: absolute;
  background: rgba(255, 255, 255, 0.95);
  border: 1px solid #ccc;
  padding: 10px;
  pointer-events: none;
  opacity: 0;
  box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
  font-size: 14px;
  transition: opacity 0.2s;
  z-index: 1000; /* <--- ADD THIS LINE */
}
```

Why this works:

z-index controls the depth of elements on your page. By default, everything sits at **0**. Giving the tooltip a massive number like **1000** guarantees it will boldly float on top of the matrix cells, the sparklines, and anything else you ever add to the page.

Save that and hover over your matrix—it should pop right to the front!

How are those mini line charts looking? Are both the red and blue lines showing up clearly inside the cells, or do we need to tweak their thickness?